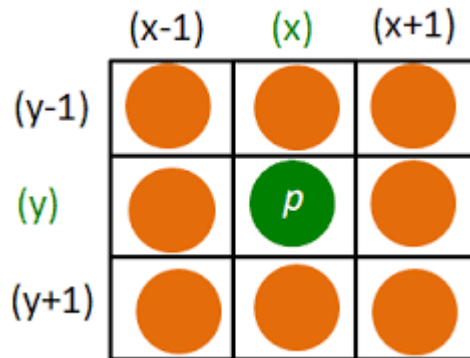


Image Enhancement

✓ Neighborhood operations processing



- Neighborhood processing enhances the image in a way doesn't possible by point processing such as:
 - Smoothing(remove spark noise)
 - Salt and pepper noise removal while preserving edges
 - Sharpening (edge detection)

Smoothing Spatial filters

Linear Filters:

- Types → Averaging filter or Weighted averaging filter
- Advantage → Remove spark noise (reduce irrelevant detail in image)
- Disadvantages → blurring edges

Different kernel examples on the weighted pixels as a function of their distance from center of mask

$$\frac{1}{16} *$$

1	2	1
2	4	2
1	2	1

3*3 Gaussian Kernel

$$\frac{1}{9} *$$

1	1	1
1	1	1
1	1	1

3*3 Averaging Kernel

$$\frac{1}{21} *$$

0	1	1	1	0
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
0	1	1	1	0

5*5 Circular Kernel

Border pixels Treatments for filters:

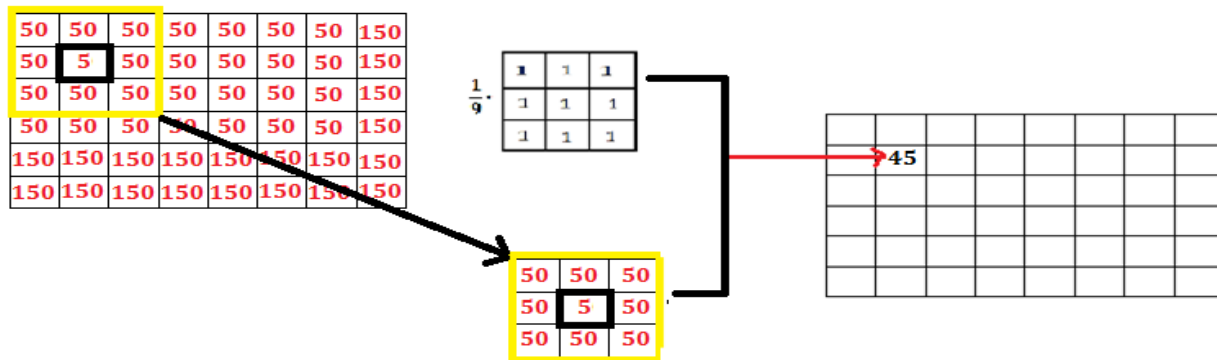
- Ignore → No processing required for those pixels.
- Reflection column and Row → the matrix rows and columns are extended out with duplication to the values of the borders.
- Null values → the matrix rows and columns are extended out with zeros.

Example: for the following image matrix apply 3*3 averaging filter, **ignoring padding and comment on the result?**

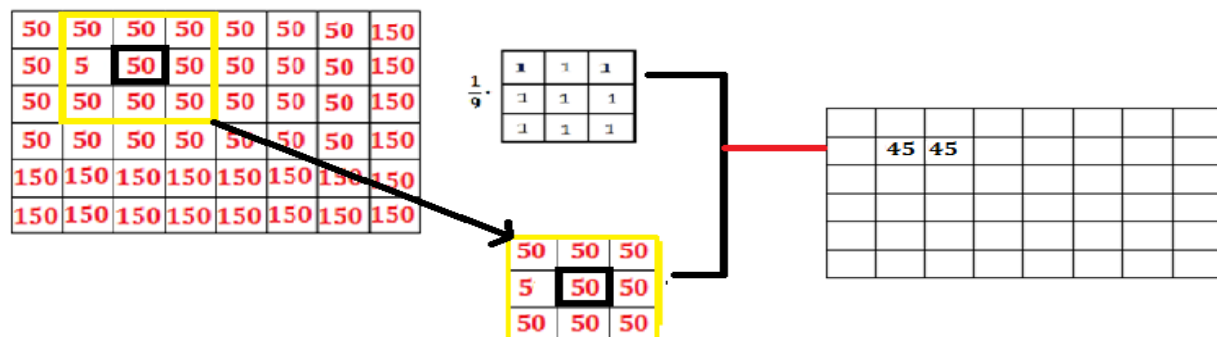
50	50	50	50	50	50	50	150
50	5	50	50	50	50	50	150
50	50	50	50	50	50	50	150
50	50	50	50	50	50	50	150
150	150	150	150	150	150	150	150
150	150	150	150	150	150	150	150

Answer:

$$\text{Output}(1,1) = \frac{50*1+50*1+50*1+50*1+50*1+50*1+50*1+50*1+5*1}{9} = \frac{50*8+5*1}{9} = 45$$



$$\text{Output}(1,2) = \frac{50*1+50*1+50*1+50*1+50*1+50*1+50*1+50*1+5*1}{9} = \frac{50*8+5*1}{9} = 45$$



50	50	50	50	50	50	50	150
50	5	50	50	50	50	50	150
50	50	50	50	50	50	50	150
50	50	50	50	50	50	50	150
150	150	150	150	150	150	150	150
150	150	150	150	150	150	150	150

original image matrix

50	50	50	50	50	50	50	150
50	45	45	50	50	50	83	150
50	45	45	50	50	50	83	150
50	83	83	83	83	83	106	150
150	117	117	117	117	117	128	150
150	150	150	150	150	150	150	150

output image matrix(enhanced image)

Although the smoothing filter enhanced the spark noise at P(1,1) which its value was 5 surrounding by neighbors values of 50 it become after filter 45, it blurt edges that transition was from (50 to 150) and became (83 117 150)

//filter2D(src, dst, ddepth, kernel);

- **Src** → A Mat object representing input image for this operation.
- **dst** → A Mat object representing output image for this operation.
- **ddepth** → representing the depth of the output image.
- **kernel** → representing the convolution kernel.

Order-Statistics Filter:

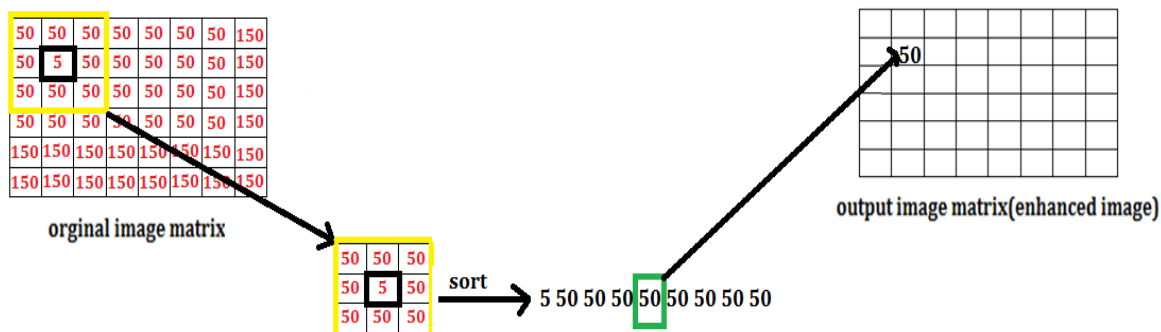
- Idea → it replaces the value of the center pixel with the value determined by the ranking result.
- Types → Min, Max or Median filter
- Advantages → remove salt and pepper noise while preserved the Edges

Example

for the following image matrix apply 3*3 Median filter, once by **ignoring padding** then **after zero padding**, comment on the result?

50	50	50	50	50	50	50	150
50	5	50	50	50	50	50	150
50	50	50	50	50	50	50	150
50	50	50	50	50	50	50	150
150	150	150	150	150	150	150	150
150	150	150	150	150	150	150	150

- Ignoring padding



50	50	50	50	50	50	50	50	150
50	5	50	50	50	50	50	50	150
50	50	50	50	50	50	50	50	150
50	50	50	50	50	50	50	50	150
150	150	150	150	150	150	150	150	150
150	150	150	150	150	150	150	150	150

original image matrix

50	50	50	50	50	50	50	50	150
50	50	50	50	50	50	50	50	150
50	50	50	50	50	50	50	50	150
50	50	50	50	50	50	50	150	150
150	150	150	150	150	150	150	150	150
150	150	150	150	150	150	150	150	150

output image matrix(enhanced image)

Median filter not only enhanced the spark noise at P(1,1) but also preserve the edges

After zero padding

- After zero padding (Null values) The matrix rows and columns are extended out with zeros.

0	0	0	0	0	0	0	0	0	0
0	50	50	50	50	50	50	50	150	0
0	50	5	50	50	50	50	50	150	0
0	50	50	50	50	50	50	50	150	0
0	50	50	50	50	50	50	50	150	0
0	150	150	150	150	150	150	150	150	0
0	150	150	150	150	150	150	150	150	0
0	0	0	0	0	0	0	0	0	0

0	0	0	0	0	0	0	0	0	0
0	50	50	50	50	50	50	50	150	0
0	50	5	50	50	50	50	50	150	0
0	50	50	50	50	50	50	50	150	0
0	50	50	50	50	50	50	50	150	0
0	150	150	150	150	150	150	150	150	0
0	150	150	150	150	150	150	150	150	0
0	0	0	0	0	0	0	0	0	0

input image matrix

0	0	0
0	50	50
0	50	5

sort

0 0 0 0 5 50 50 50

50	50	50
50	5	50
50	50	50

sort

5 50 50 50 50 50 50 50

0									
50									

output image matrix(enhanced image)

50	50	50	50	50	50	50	150
50	5	50	50	50	50	50	150
50	50	50	50	50	50	50	150
50	50	50	50	50	50	50	150
150	150	150	150	150	150	150	150
150	150	150	150	150	150	150	150

original image matrix

0	50	50	50	50	50	50	0
50	50	50	50	50	50	50	50
50	50	50	50	50	50	50	50
50	50	50	50	50	50	150	50
50	150	150	150	150	150	150	150
0	150	150	150	150	150	150	0

output image matrix(enhanced image)

//medianBlur (src, dst, ksize);

- **Src** → A Mat object representing input image for this operation.
- **dst** → A Mat object representing output image for this operation.
- **ksize**→ representing the kernel size.

```

//////////////////////////////// lab 6 //////////////////////////////////
#include<opencv2/opencv.hpp>
#include<iostream>
using namespace std;
using namespace cv;
int main()
{
    Mat src3;
    src3 = imread("x5.jpg", 0);
    namedWindow("src", 0);
    imshow("src", src3);

    Mat dst;
    Mat dstG;
    Mat kernel_T = (Mat_<float>(3, 3) << 1, 1, 1, 1, 1, 1, 1, 1, 1);
    kernel_T = kernel_T / 9;
    Mat kernel = (Mat_<float>(5, 5) << 1,1,1,1,1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1);
    kernel = kernel / 25;
    cout << endl<<kernel_T<<endl;
    filter2D(src3, dst, CV_8UC1, kernel_T);
    namedWindow("image traditional filter", 0);
    imshow("image traditional filter", dst);
    waitKey(0);

    Mat kernel_G = (Mat_<float>(3, 3) << 1, 2, 1, 2, 4, 2, 1, 2, 1);
    kernel_G = kernel_G / 16;
    filter2D(src3, dstG, CV_8UC1, kernel);
    namedWindow("image traditional filter2", 0);
    imshow("image traditional filter2", dstG);
    waitKey(0);

    Mat srcM, dstM;
    srcM = imread("med2.jpg", 0);
    namedWindow("before median filter", 0);
    imshow("before median filter", srcM);
    medianBlur(srcM, dstM, 3);
    namedWindow("median filter", 0);
    imshow("median filter", dstM);
    waitKey(0);
}

```